

Foundations of Linear Algebra

for NLP · CS6120 recitation, session 2

why it matters, the intuition, the math, and the numpy for each idea.

colour key



input



second input



result



degenerate case

why linear algebra?

Every model in this course, from a text classifier to word embeddings to attention, is built from vectors and matrices being multiplied. Linear algebra is the notation these models are written in.

Once the notation is familiar, they read less like magic and more like ordinary arithmetic. These slides cover the pieces that come up again and again, with the intuition, the math, and the numpy for each.

vectors and matrices: the notation every model is written in

two useful framings

WHERE DOES A MATRIX COME FROM?

- solving a system like $2x + y = 5$, $x - y = 1$ is already linear algebra.
- collect the coefficients into a grid and you have a matrix, a compact way to hold a whole system at once instead of one line at a time.

WHY VECTORS, IF WE HAVE MATRICES?

- a vector is not a separate object from a matrix. an n-dimensional vector is a matrix with n rows and one column, a column vector.
- row versus column is about shape, not kind. it only depends on how the numbers are laid out.

a vector = a matrix with one column. every topic here is one idea seen at different shapes.

vectors

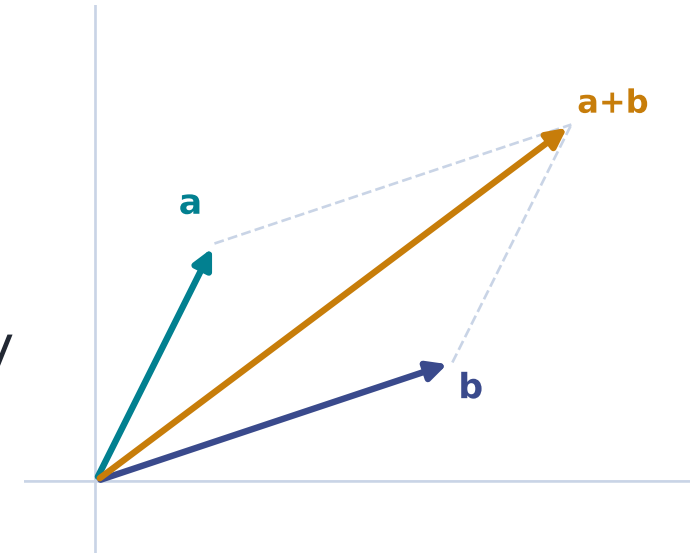
WHY

A vector is both a list of numbers and an arrow from the origin, the same object seen two ways. The length of the list is its dimension. In NLP each word is represented as a vector, so two words count as similar when their arrows point in nearly the same direction.

THE MATH

$a + b$: add position by position; αa : scale every entry

$a=[1,2]$, $b=[3,1]$: $a+b=[4,3]$, $2a=[2,4]$



IN CODE

```
a = np.array([1, 2]); b = np.array([3, 1])
a + b           # [4 3]      2 * a   # [2 4]
```

dot product

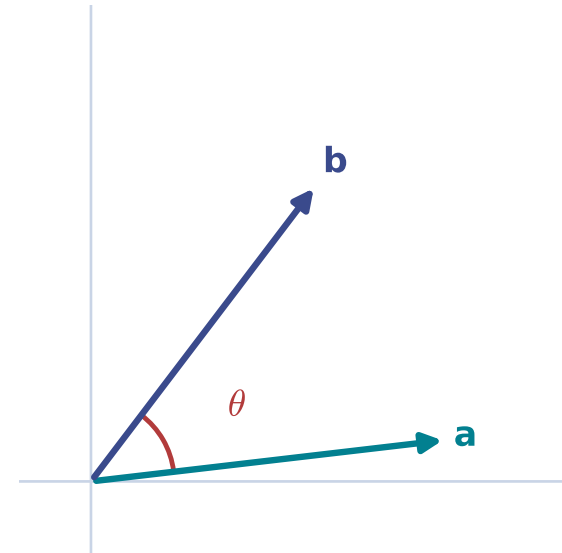
WHY

The dot product measures how much two vectors point the same way. Multiply the matching entries and add them into one number. It is positive when the vectors are aligned, zero when they are perpendicular, and negative when they point opposite ways.

THE MATH

$$\mathbf{a} \cdot \mathbf{b} = \sum_j a_j b_j$$

$$[1,2,3] \cdot [4,5,6] = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$



IN CODE

```
a @ b          # 32    (positive = aligned, 0 = perpendicular)
```

cross product

WHY

The cross product takes two vectors and returns a third that is perpendicular to both, like the normal to a surface. It is defined in 3D and comes up mostly in graphics and physics rather than NLP, but it completes the set of vector operations.

THE MATH

$$\mathbf{a} \times \mathbf{b} = [a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1]$$

$$[1,0,0] \times [0,1,0] = [0,0,1] \quad (\text{perpendicular to both})$$

IN CODE

```
np.cross([1,0,0], [0,1,0])    # [0 0 1]
```

length (norm) and unit vectors

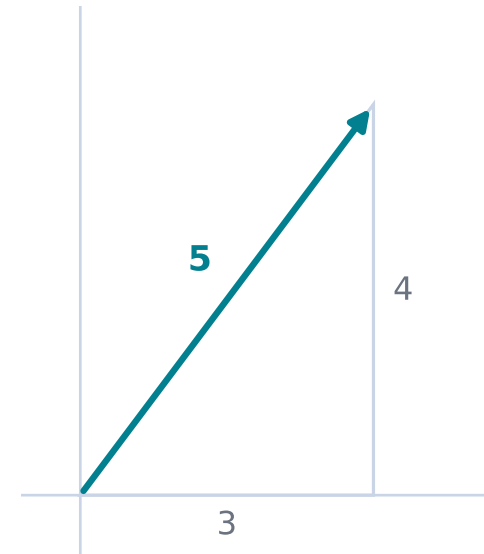
WHY

The norm is the length of a vector, measured with Pythagoras: square the entries, add, take the square root. Dividing a vector by its norm keeps its direction but sets its length to 1, a unit vector. Directions are compared after normalizing so length does not distort the result.

THE MATH

$$\|\mathbf{v}\| = \sqrt{\sum_j v_j^2}, \quad \hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\|$$

$$\|[3,4]\| = \sqrt{9+16} = 5, \quad \text{so } \mathbf{v}/5 = [0.6, 0.8]$$



IN CODE

```
np.linalg.norm([3,4])      # 5.0
v / np.linalg.norm(v)      # length 1
```

cosine similarity

WHY

Cosine similarity compares two vectors by direction alone, ignoring their lengths. Normalize both to unit length and take the dot product, which gives the cosine of the angle between them: 1 when they point the same way, 0 when perpendicular, negative when opposed. Because length is dropped, a short text and a long one on the same topic still score as similar.

THE MATH

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

$\mathbf{a}=[1,0]$, $\mathbf{b}=[1,1]$: $1 / (1 \cdot 1.41) = 0.71$ (a 45° angle)

in NLP this is the standard measure of similarity between words or documents.

IN CODE

```
(a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

matrices

WHY

A matrix is a grid of numbers, which can also be read as a stack of vectors or as a function that maps one vector to another. Its shape is rows by columns, and the entry $M(i,j)$ is the value at row i , column j .

THE MATH

M has shape $\text{rows} \times \text{columns}$; $M_{ij} = \text{row } i, \text{ column } j$

`M = [[1,2,3],[4,5,6]]` has shape `(2, 3)`; entry `row2,col3 = 6`

IN CODE

```
M = np.array([[1,2,3],[4,5,6]])  
M.shape      # (2, 3)
```

matrix times vector

WHY

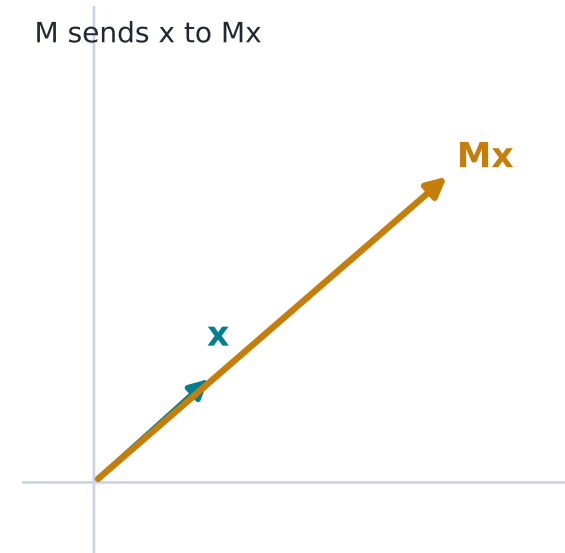
Multiplying a matrix by a vector produces a new vector. Each output entry is one row of the matrix dotted with the input. Geometrically the matrix stretches and rotates space, and the result is where the input vector lands.

THE MATH

$$(M\mathbf{x})_i = \sum_j M_{ij}x_j$$

$$\text{row1} \cdot \mathbf{x} = 1 \cdot 1 + 0 \cdot 2 + 2 \cdot 1 = 3$$

$$\text{row2} \cdot \mathbf{x} = 0 \cdot 1 + 3 \cdot 2 + 1 \cdot 1 = 7 \quad \rightarrow [3, 7]$$



IN CODE

```
M @ x      # [3 7]  (one dot product per row)
```

matrix times matrix

WHY

Multiplying two matrices composes their transformations into one, or applies a transformation to a whole set of vectors at once. Each output entry is a row of the first matrix dotted with a column of the second. The inner dimensions must match for the product to be defined.

THE MATH

$$(AB)_{ij} = \sum_k A_{ik} B_{kj} , \quad (n, k) \times (k, m) \rightarrow (n, m)$$

$$\text{entry}(1,1) = 1 \cdot 5 + 2 \cdot 7 = 19 \quad \text{entry}(2,1) = 3 \cdot 5 + 4 \cdot 7 = 43$$

$$[[1,2],[3,4]] @ [[5,6],[7,8]] = [[19,22],[43,50]] \quad (\text{one dot per row-col pair})$$

IN CODE

```
A @ B          # row i of A, dotted with column j of B
```

transpose

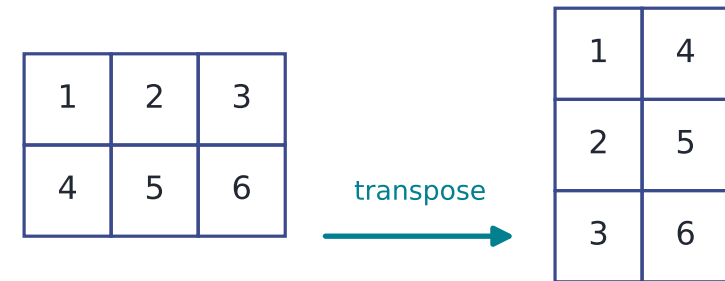
WHY

The transpose flips a matrix across its diagonal, turning rows into columns. It is used to line a row up against a column so the shapes match for multiplication. An m by n matrix becomes n by m.

THE MATH

$$(M^{\top})_{ij} = M_{ji}$$

`[[1,2,3],[4,5,6]]` (2x3) -> `[[1,4],[2,5],[3,6]]` (3x2)



IN CODE

```
M.T          # rows and columns swapped
```

special matrices

WHY

A few matrices appear often enough to have names. The identity leaves a vector unchanged, a diagonal matrix scales each axis independently, a symmetric matrix equals its own transpose, and an orthogonal matrix rotates without changing lengths.

THE MATH

I : identity · diagonal · symmetric ($M = M^T$) · orthogonal

IN CODE

```
np.eye(2)      # identity  [[1,0],[0,1]]
np.diag([2,3]) # diagonal, scales each axis
```

determinant

WHY

The determinant is a single number describing how a matrix scales area or volume, and whether its transformation can be reversed. A determinant of zero means the matrix collapses space onto a lower dimension, which cannot be undone, so the matrix has no inverse.

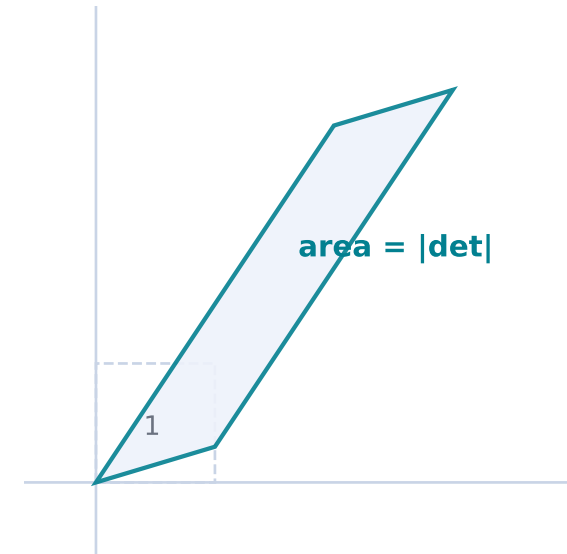
THE MATH

$$\det = ad - bc \quad (\text{for a } 2 \times 2 \text{ matrix})$$

$$\det(\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}) = 1 \cdot 4 - 2 \cdot 3 = -2 \quad \text{nonzero, so invertible}$$

$$\det(\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}) = 1 \cdot 4 - 2 \cdot 2 = 0 \quad \text{rows dependent, no inverse}$$

teal = holds red = degenerate case



IN CODE

```
np.linalg.det([[1,2],[3,4]])    # -2.0
```

inverse

WHY

The inverse of a matrix reverses its transformation: applying a matrix and then its inverse returns the original vector. An inverse exists only when the matrix loses no information, that is, when its determinant is nonzero.

THE MATH

$$MM^{-1} = I \quad (\text{exists only when } \det \neq 0)$$

$$M = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \rightarrow M^{-1} = \begin{bmatrix} -2 & 1 \\ 1.5 & -0.5 \end{bmatrix}; \quad M \cdot M^{-1} = I$$

IN CODE

```
np.linalg.inv([[1,2],[3,4]])
```

rank and linear independence

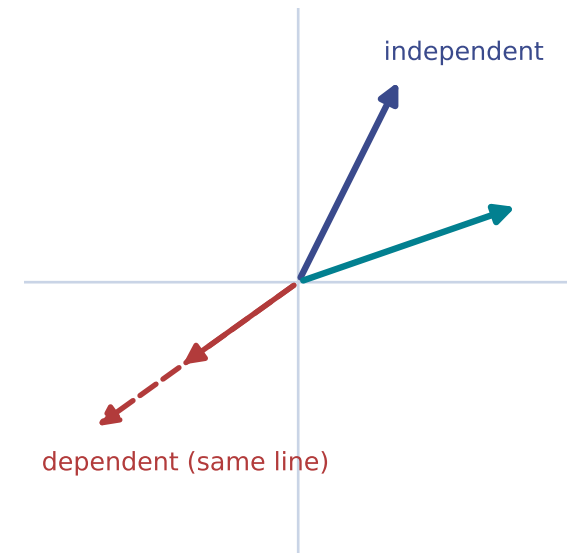
WHY

The rank is the number of independent directions spanned by a matrix's rows or columns. A row that is a combination of others adds no new direction, so duplicates and scaled copies do not raise it. It measures the true, non-redundant size of the data.

THE MATH

rank = number of independent rows (or columns)

$[[1,2],[2,4]]$: $\text{row2} = 2 \cdot \text{row1} \rightarrow \text{rank } 1$. $[[1,0],[0,1]] \rightarrow \text{rank } 2$.



IN CODE

```
np.linalg.matrix_rank([[1,2],[2,4]])    # 1
```

null space and range

WHY

The range of a matrix is the set of all vectors it can produce as outputs. The null space is the set of all inputs it maps to zero. Together they describe everything the matrix can reach and everything it discards.

THE MATH

range: all $M\mathbf{x}$ · null space: all $\mathbf{x}$ with $M\mathbf{x} = \mathbf{0}$

$M = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$: $\mathbf{x} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$ gives $M \cdot \mathbf{x} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, so $\begin{bmatrix} 2 \\ -1 \end{bmatrix}$ is in the null space.

IN CODE

```
# rank + nullity = number of columns
np.linalg.matrix_rank(M)    # 1  -> nullity 1
```

eigenvectors and eigenvalues

WHY

Most vectors change direction when a matrix is applied to them. An eigenvector is one that keeps its direction and is only scaled, and the scale factor is its eigenvalue λ . The defining condition is $Mv = \lambda v$.

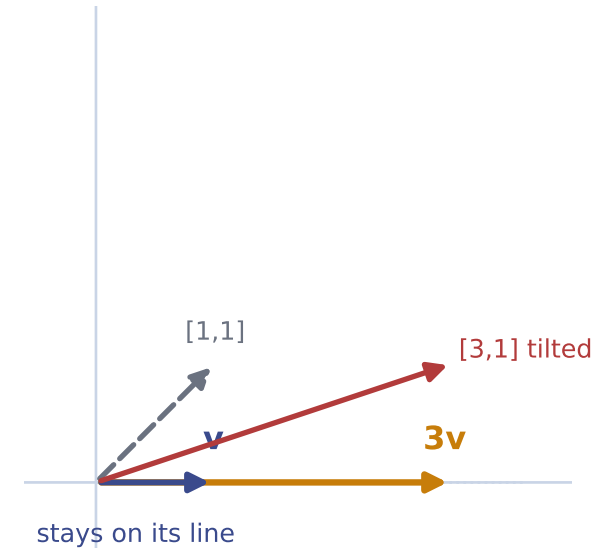
THE MATH

$$M\mathbf{v} = \lambda\mathbf{v}$$

$M = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$: $[1,0] \rightarrow 3 \cdot [1,0] \lambda=3$, $[0,1] \rightarrow 1 \cdot [0,1] \lambda=1$

$[1,1] \rightarrow [3,1]$ tilts off its line, not an eigenvector

PCA and SVD keep the top eigen-directions to compress embeddings.



IN CODE

```
np.linalg.eig([[3,0],[0,1]])    # eigenvalues [3, 1]
```

sums and products (Σ and Π)

WHY

Sigma denotes the sum of a list of terms and Pi denotes their product. Both apply one operation across every element and combine the results into a single value.

THE MATH

$$\sum_{i=1}^N x_i = x_1 + x_2 + \dots + x_N$$

$$\prod_{i=1}^N x_i = x_1 \cdot x_2 \cdots x_N$$

$x = [2, 3, 5]$: sum = 10, product = 30

IN CODE

```
np.sum(x)      # 10  
np.prod(x)     # 30
```

properties of logs

WHY

Logarithms convert multiplication into addition. Multiplying many probabilities drives the result toward zero and causes numerical underflow; taking the log turns that product into a sum that is stable to compute. This is why log terms appear in loss functions.

THE MATH

$$\log(ab) = \log a + \log b$$

$$\log \prod_i x_i = \sum_i \log x_i, \quad \log e^x = x$$

$$\log(2 \cdot 4) = \log 2 + \log 4; \quad \log(e^3) = 3$$

a product of probabilities becomes a sum of logs. that is why cross-entropy uses logs.

IN CODE

```
np.log(a*b) == np.log(a) + np.log(b)
```

additional resources

For deeper study and visual intuition, these are good next steps.

- **3Blue1Brown, Essence of Linear Algebra**

youtube.com/playlist?list=PLZHQ0b0WTQDPD3MizzM2xVFitgF8hE_ab

- **NumPy linear algebra reference**

numpy.org/doc/stable/reference/routines.linalg.html

recap

vectors, dot products, norms, cosine, matrices and their products,

transpose, determinant, inverse, rank, null space, and eigenvectors, the core toolkit for the rest of the course. next in the math sequence is multivariate calculus, the gradients used to train these models.